

Testril demo plan

Review summary · 3 October 2026

Purpose

Help engineers evaluate Testril through runnable demos: sub-cent payments, materialization earnings, and result verification now; function creation when implemented.

Testril charges for preparing and storing computed data (**materialization**), then reading it. The materialization payer can earn from later reads. **Provenance** records source data; checking the calculation requires independent verification.

Small payments should be central to the demos. Testril is built around tiny purchases; conventional APIs and billing can be added above them. Competitors are adding machine payments to existing APIs. Nansen lists 1¢/5¢ calls; Allium lists 1¢ SQL submission and 2¢/3¢ data calls. [Sources and limits of this comparison](#).

First release

P0. Sub-cent data reads. Use the existing transfer-volume function to check token activity in short block ranges. Choose each next range from the previous result. Show the result, charge, and remaining budget at every step. Fifty 100-block cached reads would cost **\$0.006 (0.6¢)** at the observed rates. Use as many checks as the task needs; show preparation costs separately.

P1. Treasury outflow analysis. Answer “Where did this treasury's USDC go?” for one token, chain, and block range. Show incoming and outgoing transfers, counterparties, and the balance change. Use supplied team addresses to separate internal transfers. Export the result and a runnable client example. Requires a new **transfer-edge function**, which returns amounts between pairs of addresses, and exact balance snapshots.

P2. Materialization earnings. A second application pays to read data prepared for P1. Show the original payer's earnings before and after that read, alongside both customers' costs. Use separate accounts. Verify an actual payout before showing one. Label traffic funded by the demo operator; it demonstrates accounting, not customer demand.

P3. Result verification. Add verification and export to P1. Fetch the source logs and state at the recorded blocks, recalculate the result, then deliberately alter a displayed amount and show the failed comparison. A source-block digest alone cannot verify the arithmetic.

Optional additions before function creation

P4. Pool activity and liquidity: compare one Uniswap v3 pool's swaps and active liquidity; requires two functions. **P5. REST access and card billing:** add an API-key gateway that handles Testril payments. Choose between these based on user trials.

After function creation is implemented

G1. Generate a deposit classification function. Ask Testril to group vault deposits by each owner's token balance at the previous block. Show generated code, tests, costs, and results, then reuse it from another client. Requires vault functions and generation support.

G2. Generate a repeat-deposit function. Define and revise a rule for deposits repeated on a later UTC day within seven days. Show incomplete observation periods separately. Requires stored state, history, and timestamps as well as generation.

G3. Generate a function and earn from reused data. Combine G1 and P2 once both work. Materialization holders earn; author royalties would require another feature.

Wallet setup and spending limits

Offer a recording, a sponsor-funded live trial, and a user-funded wallet option. Let visitors run a useful example before setting up a wallet. The wallet authorizes a sequence of small purchases within a budget and receives eligible materialization earnings.

Enforce spending limits in client code outside the LLM. Show escrow deposits separately from read charges and unused funds. Use a wallet or server-side signer; the public demo must not ask for a private key. A sponsored trial leaves the sponsor holding any resulting materialization rights unless ownership transfer is implemented.

Card billing and REST proposal

Build a small gateway for customers who want API keys and card payments. It should buy through Testril's paid MCP interface and return the same data and provenance as direct access. Keep sub-cent precision in usage records and combine charges for billing. Do not round each underlying read up to one cent.

Start with prepaid credit and a spending cap. Price subscriptions after measuring usage and operating costs. For the first version, the gateway funds materialization and holds the right to earnings. Any customer rebate must be recorded explicitly as service credit. Serving a gateway cache hit does not create another Testril reward.

Implementation and verification

Build **P0–P3 first**. The preliminary estimate is **11–21 engineer-days**, including the transfer-edge function if the current engine supports it. Backend engine changes are additional work. The detailed plan assigns work and completion criteria; the function requirements specify inputs, outputs, and test cases.

The inspected endpoint lists three functions and Ethereum, Arbitrum, and Base Sepolia data support. Paid execution, payment network, latency, and payouts still need verification. At observed rates, preparing 1,000 blocks costs **\$0.12** and reading that cached range costs **\$0.00102**. These are data charges; track LLM and reference-RPC costs separately. The fifty-read example is arithmetic, not a measured competitor benchmark. Prefer a bulk read when the whole required range is known.

For each working demo, publish runnable code, a short recording, and reproducible output. Test setup with five developers. A proposed reason to continue is that three finish unaided, two return with another task, and one requests an integration. Record failures and wallet setup abandonment.

Decisions needed: choose the first real treasury case; assign the transfer-edge implementation; choose the demo payment network and budget; agree who receives gateway earnings; define the tests required before demonstrating function creation.

[Detailed plan](#) · [Function requirements](#) · [Research notes](#)